



Assignment: 02

Name: Tuba Jan

Student ID: CSC-25F-602

Subject: Object Oriented Programming

Techer Name: Sir Engr .Majid Kaleem



Sindh Madressatul Islam University

Q-1 (a): Define modularity and reusability in the context of software application development.

- **Modularity:** Breaking a large, complex software application into smaller, independent pieces called "modules." Each module handles a specific task.
- **Reusability:** Writing code once and using it in different parts of the same application or even in entirely different projects, without modifying it.

b) How OOP Principles Contribute

- **Classes & Objects:** A **Class** is a blueprint (like a factory design). An **Object** is a created instance. They group related data and behavior together, making code organized (**Modularity**).
- **Encapsulation:** Hiding internal details and exposing only what is necessary. It keeps code safe and makes modules independent.
- **Inheritance:** Allows a new class to use the code of an existing class. This is a core way to achieve **Reusability**.
- **Polymorphism:** Allows different objects to be treated in the same way (e.g., a "Shape" class can have a "draw" method that works differently for "Circle" and "Square"). It makes code flexible and easier to extend.

QUESTION 2

a) Provide at least two real-world examples where modularity and reusability improve software quality.

Payment Gateway: A "Payment Module" can be written once and **reused** for Credit Cards, PayPal, or Crypto. If the logic changes, you only update that specific module.

1. **UI Component Libraries:** In web apps, a "Button" or "Navbar" component is a **module**. It is created once and **reused** across hundreds of different pages to maintain consistency and quality.

b) Code Concept (Simple Example)

i. Two Classes with Clear Responsibilities

We separate the "Data" from the "Action":

- **User:** Responsible for holding user information.
- **EmailService:** Responsible only for sending notifications.

```
TypeScript
class User {
  constructor(public name: string, public email: string) {}
}

class EmailService {
  sendWelcome(user: User) {
    console.log(`Sending email to ${user.email}`);
  }
}
```

ii. Use of Inheritance and Interfaces

We use an **Interface** to define a contract and **Inheritance** to reuse logic.

```
TypeScript
// Interface (The Contract)
interface Shape {
```

```

    getArea(): number;
}

// Parent Class (Base Logic)
class BasicShape {
    color: string = "Red";
}

// Inheritance + Interface
class Circle extends BasicShape implements Shape {
    constructor(private radius: number) { super(); }

    getArea() {
        return Math.PI * this.radius ** 2;
    }
}

```

- **Modularity:** The `Shape` interface ensures any new shape (Square, Triangle) fits into the system.
- **Reusability:** `Circle` reuses the `color` property from `BasicShape`.

Question 3: Class Types in OOP (BCE Pattern)

a) Definitions

- **Boundary Classes:** The "Interface." They handle interaction between the system and external actors (e.g., `LoginForm`, `API_Gateway`).
- **Control Classes:** The "Brain." They coordinate logic and flow between boundaries and entities (e.g., `LoginController`, `OrderProcessor`).
- **Entity Classes:** The "Data." They represent persistent information or real-world objects (e.g., `User`, `Product`, `Invoice`).

b) Contribution to Modularity and Reusability

- **Modularity:** By separating the UI (Boundary) from the Logic (Control), you can change the visual design without breaking the underlying math.

- **Reusability:** Entity classes (like User) are independent; they can be plugged into different Control modules (like Payroll or Security) without modification.

QUESTION 4:



